

In interactive GUI programs, this model no longer works. GUIs introduce a new model of computing known as the *event-driven* model. Event-driven programs don't rely on `input()` and `print()`; instead, the user interacts with elements in a window at will using a keyboard and/or mouse or other pointing device. They may be able to click various buttons or icons, make selections from menus, provide input in text fields, or give commands via clicks or key presses to control some avatar in the window.

NOTE

Calls to `print()` can still be highly useful for debugging, when used to write out intermediate results.

Central to event-driven programming is the concept of an *event*. Events are difficult to define and are best described with examples, such as a mouse click and a key press (each of which is actually made up of two events: mouse down and mouse up and key down and key up, respectively). Here is my working definition.

event

Something that happens while your program is running that your program wants to or needs to respond to. Most events are generated by user actions.

An event-driven GUI program runs constantly in an infinite loop. Each time through the loop, the program checks for any new events it needs to react to and executes appropriate code to handle those events. Also, each time through the loop, the program needs to redraw all the elements in the window to update what the user sees.

For example, say we have a simple GUI program that displays two buttons: Bark and Meow. When clicked, the Bark button plays a sound of a dog barking and the Meow button plays a sound of a cat meowing ([Figure 5-5](#)).